```
"""
Outfit Finder - Trouve des références pas chères à partir d'une photo d'outfit
et génère un moodboard.

Lancer avec : streamlit run app.py
"""

import base64
import io
import json
import os

import requests
import streamlit as st
from PIL import Image, ImageDraw, ImageFont

# ---------------------------------------------------------------------------
# CONFIG
# ---------------------------------------------------------------------------
st.set_page_config(page_title="Outfit Finder", page_icon="👗", layout="wide")

MOODBOARD_WIDTH = 1200
THUMB_SIZE = (260, 260)
BG_COLOR = (245, 245, 245)


# ---------------------------------------------------------------------------
# ETAPE 1 : Identification des vêtements dans la photo (via Claude vision)
# ---------------------------------------------------------------------------
def identify_garments(image_bytes: bytes, anthropic_api_key: str) -> list[dict]:
    """
    Envoie la photo à l'API Claude (vision) et récupère une liste de
    vêtements identifiés avec une description utile pour la recherche.
    """
    b64_image = base64.b64encode(image_bytes).decode("utf-8")

    prompt = (
        "Regarde cette photo d'un outfit. Identifie chaque pièce vestimentaire "
        "visible (haut, bas, chaussures, sac, accessoires, etc). "
        "Réponds UNIQUEMENT avec un JSON valide, sans texte autour, sous la forme
        '[{"item": "nom court", "search_query": "description précise pour '
        'un moteur de recherche shopping (couleur, matière, coupe, style)"}]'
    )
```

```
    response = requests.post(
        "https://api.anthropic.com/v1/messages",
        headers={
            "x-api-key": anthropic_api_key,
            "anthropic-version": "2023-06-01",
            "content-type": "application/json",
        },
        json={
            "model": "claude-sonnet-4-6",
            "max_tokens": 1000,
            "messages": [
                {
                    "role": "user",
                    "content": [
                        {
                            "type": "image",
                            "source": {
                                "type": "base64",
                                "media_type": "image/jpeg",
                                "data": b64_image,
                            },
                        },
                        {"type": "text", "text": prompt},
                    ],
                }
            ],
        },
        timeout=60,
    )
    response.raise_for_status()
    data = response.json()
    text = "".join(
        block.get("text", "") for block in data.get("content", []) if block.get("
    )
    text = text.strip().removeprefix("```json").removeprefix("```").removesuffix(
    return json.loads(text)


# ---------------------------------------------------------------------------
# ETAPE 2 : Recherche de produits pas chers (via SerpAPI - Google Shopping)
# ---------------------------------------------------------------------------
def search_cheap_products(query: str, serpapi_key: str, max_results: int = 3) ->
    """
    Cherche des produits correspondant à la requête, triés par prix croissant.
    Retourne une liste de dicts : title, price, thumbnail, link
    """
    resp = requests.get(
```

```
        "https://serpapi.com/search.json",
        params={
            "engine": "google_shopping",
            "q": query,
            "api_key": serpapi_key,
            "hl": "fr",
            "gl": "fr",
        },
        timeout=30,
    )
    resp.raise_for_status()
    results = resp.json().get("shopping_results", [])

    def price_value(item):
        raw = item.get("extracted_price")
        return raw if raw is not None else float("inf")

    results_sorted = sorted(results, key=price_value)
    products = []
    for item in results_sorted[:max_results]:
        products.append(
            {
                "title": item.get("title", "Sans titre"),
                "price": item.get("price", "Prix inconnu"),
                "thumbnail": item.get("thumbnail"),
                "link": item.get("link") or item.get("product_link", "#"),
                "source": item.get("source", ""),
            }
        )
    return products


# ---------------------------------------------------------------------------
# ETAPE 3 : Génération du moodboard (Pillow)
# ---------------------------------------------------------------------------
def build_moodboard(original_image: Image.Image, items_with_products: list[dict])
    """
    Compose une planche : photo originale à gauche, grille de produits trouvés
    à droite avec titre + prix.
    """
    try:
        font = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans.tt
        font_bold = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSa
    except Exception:
        font = ImageFont.load_default()
        font_bold = font
```

```
all_products = [p for it in items_with_products for p in it["products"]]
n_products = max(len(all_products), 1)
cols = 3
rows = (n_products + cols - 1) // cols

orig_w = 420
orig_h = int(original_image.height * (orig_w / original_image.width))
original_resized = original_image.resize((orig_w, orig_h))

grid_w = cols * (THUMB_SIZE[0] + 20) + 20
grid_h = rows * (THUMB_SIZE[1] + 60) + 20

canvas_w = orig_w + grid_w + 60
canvas_h = max(orig_h + 60, grid_h) + 80
canvas = Image.new("RGB", (canvas_w, canvas_h), BG_COLOR)
draw = ImageDraw.Draw(canvas)

draw.text((20, 15), "Ton outfit", font=font_bold, fill=(20, 20, 20))
canvas.paste(original_resized, (20, 50))

grid_x0 = orig_w + 40
draw.text((grid_x0, 15), "Références pas chères", font=font_bold, fill=(20, 2

x, y = grid_x0, 50
for i, product in enumerate(all_products):
    col = i % cols
    if col == 0 and i != 0:
        y += THUMB_SIZE[1] + 60
    x = grid_x0 + col * (THUMB_SIZE[0] + 20)

    try:
        thumb_resp = requests.get(product["thumbnail"], timeout=15)
        thumb = Image.open(io.BytesIO(thumb_resp.content)).convert("RGB")
        thumb.thumbnail(THUMB_SIZE)
    except Exception:
        thumb = Image.new("RGB", THUMB_SIZE, (220, 220, 220))

    paste_x = x + (THUMB_SIZE[0] - thumb.width) // 2
    canvas.paste(thumb, (paste_x, y))

    title = product["title"]
    title = title if len(title) <= 32 else title[:29] + "..."
    draw.text((x, y + THUMB_SIZE[1] + 4), title, font=font, fill=(40, 40, 40)
    draw.text(
        (x, y + THUMB_SIZE[1] + 24),
        f"{product['price']}",
        font=font_bold,
```

```
            fill=(0, 120, 60),
        )

    return canvas


# ---------------------------------------------------------------------------
# UI STREAMLIT
# ---------------------------------------------------------------------------
def main():
    st.title("👗 Outfit Finder")
    st.caption("Importe une photo d'outfit → trouve des références pas chères → g

    with st.sidebar:
        st.header("🔑 Clés API")
        anthropic_key = st.text_input(
            "Clé API Anthropic (identification des vêtements)",
            type="password",
            value=os.environ.get("ANTHROPIC_API_KEY", ""),
        )
        serpapi_key = st.text_input(
            "Clé API SerpAPI (recherche de produits)",
            type="password",
            value=os.environ.get("SERPAPI_KEY", ""),
        )
        max_per_item = st.slider("Nb de références par vêtement", 1, 5, 3)
        st.markdown("---")
        st.markdown(
            "**Où obtenir les clés ?**\n\n"
            "- Anthropic : [console.anthropic.com](https://console.anthropic.com)
            "- SerpAPI : [serpapi.com](https://serpapi.com) (offre gratuite dispo
        )

    uploaded_file = st.file_uploader("📸 Importe une photo de ton outfit", type=[

    if uploaded_file is None:
        st.info("Importe une image pour commencer.")
        return

    image_bytes = uploaded_file.read()
    original_image = Image.open(io.BytesIO(image_bytes)).convert("RGB")

    st.image(original_image, caption="Photo importée", width=350)

    if st.button("✨ Générer le moodboard", type="primary"):
        if not anthropic_key or not serpapi_key:
            st.error("Merci de renseigner les deux clés API dans la barre latéral
```

```
    return

with st.spinner("Identification des vêtements..."):
    try:
        garments = identify_garments(image_bytes, anthropic_key)
    except Exception as e:
        st.error(f"Erreur lors de l'identification : {e}")
        return

if not garments:
    st.warning("Aucun vêtement détecté.")
    return

st.subheader("Vêtements identifiés")
items_with_products = []
progress = st.progress(0)
for i, garment in enumerate(garments):
    st.write(f"**{garment['item']}** — recherche : _{garment['search_quer
    try:
        products = search_cheap_products(
            garment["search_query"], serpapi_key, max_results=max_per_ite
        )
    except Exception as e:
        st.warning(f"Recherche échouée pour {garment['item']} : {e}")
        products = []

    cols = st.columns(len(products)) if products else []
    for col, product in zip(cols, products):
        with col:
            if product["thumbnail"]:
                st.image(product["thumbnail"], use_container_width=True)
            st.markdown(f"**{product['price']}**")
            st.caption(product["title"])
            st.markdown(f"[Voir l'article]({product['link']})")

    items_with_products.append({"item": garment["item"], "products": prod
    progress.progress((i + 1) / len(garments))

st.subheader("🖼️ Ton moodboard")
with st.spinner("Composition du moodboard..."):
    moodboard = build_moodboard(original_image, items_with_products)

st.image(moodboard, use_container_width=True)

buf = io.BytesIO()
moodboard.save(buf, format="PNG")
st.download_button(
```

```
            "⬇️ Télécharger le moodboard",
            data=buf.getvalue(),
            file_name="moodboard.png",
            mime="image/png",
        )


if __name__ == "__main__":
    main()
```